

Data-Driven Computer Animation

Learning Real-time Character Controllers from Motion Capture Data



Taku Komura

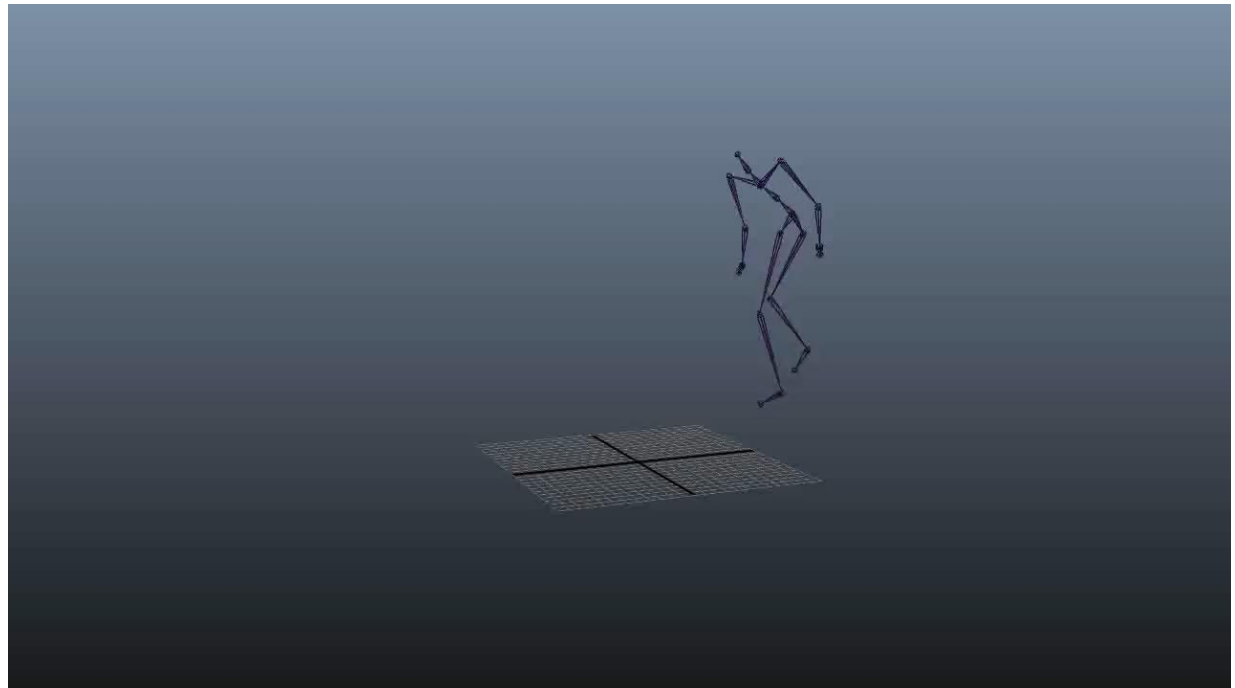
Goal

- Building interactive character controllers that follows the instruction of the user
- Synthesizing realistic and smooth character motions
- Learning from a large motion capture data set



Learning from Motion Capture Data

We wish to learn from unstructured motion capture data



Classic Data-Driven Character Controllers

Classic approaches:

- Finite State Machine, Motion graphs
 - A graph structure computed from the motion capture data
 - Segment and classify the motions
 - Decide the connectivity
 - Launching a motion given the user instruction

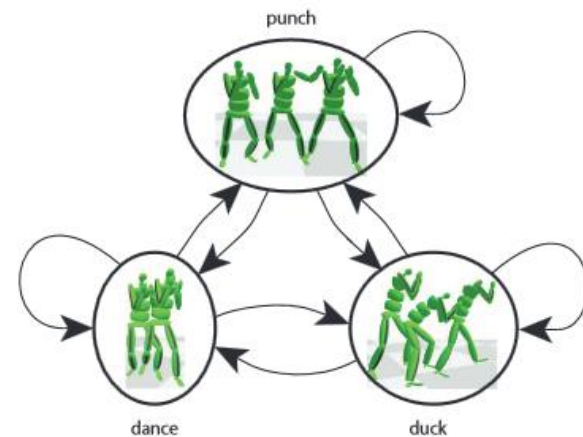
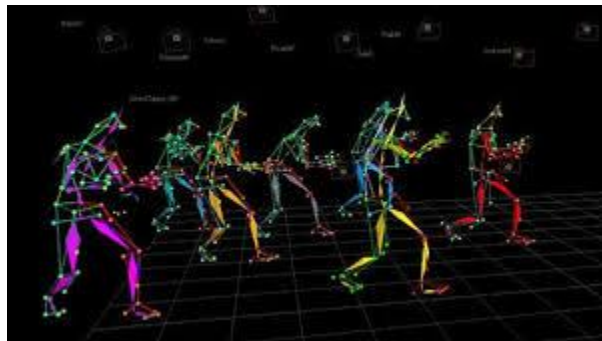


Figure 6: A boxing graph.

Classic Data-Driven Character Controllers

- Issues:
 - Many procedures need to be done manually or fixed manually
 - E.g., segmentation, labelling, etc.
 - Requires keeping all the motion in the memory

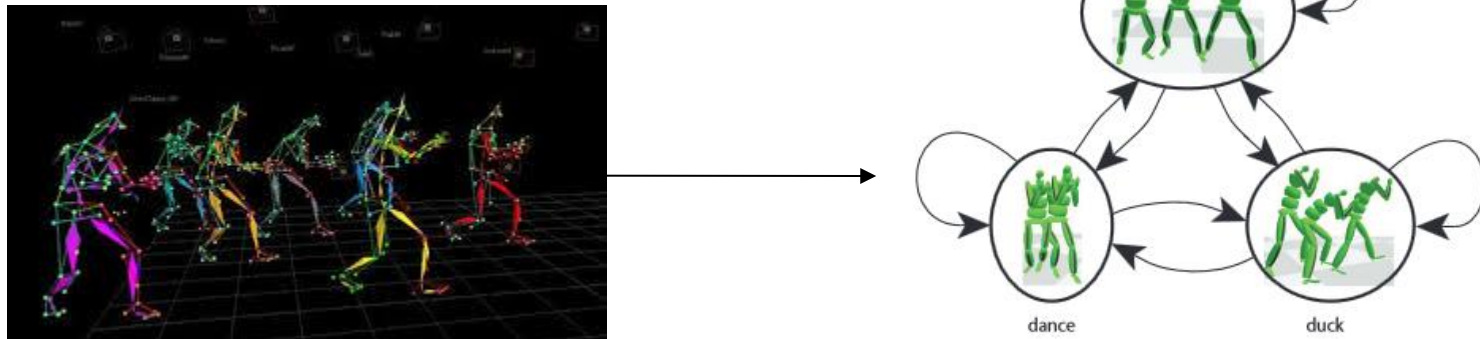
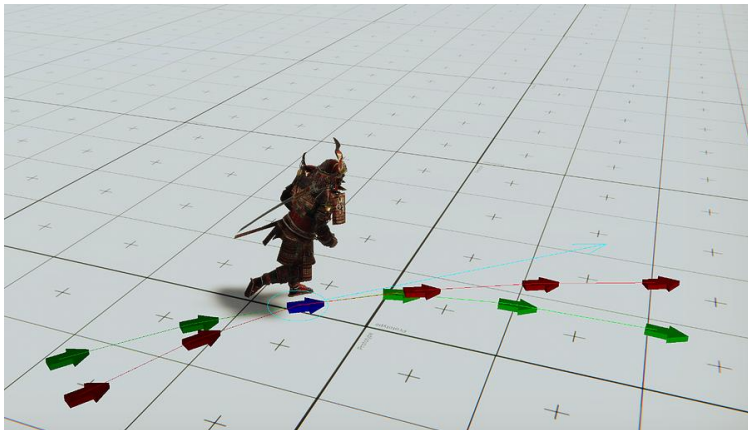


Figure 6: A boxing graph.

Motion Matching (revisited)

- A technique developed by Ubisoft Montreal
- Define a feature, search a motion in the database that matches the current pose and user instruction and then plays it
- Current state-of-the-art in the computer game industry



Clavet 2016, "Motion Matching and The Road to Next-Gen Animation"
Holden et al. 2020, "Learned Motion Matching"

Motion Matching

- Indeed, motion matching searches for the motion online, instead of pre-computing a state machine
- Easier to enhance the dataset as no precomputation is needed
- Still can be slower when the database grows
- Also cannot search for every frame – the character can get stuck in the current frame

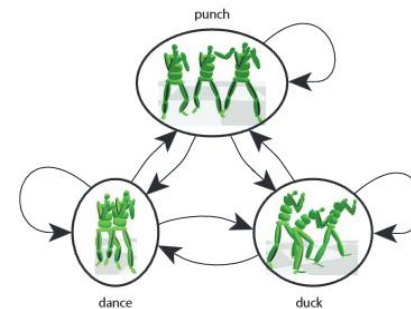
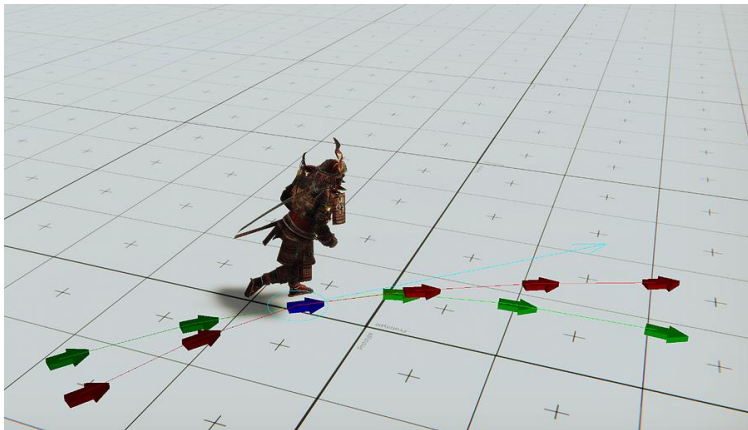
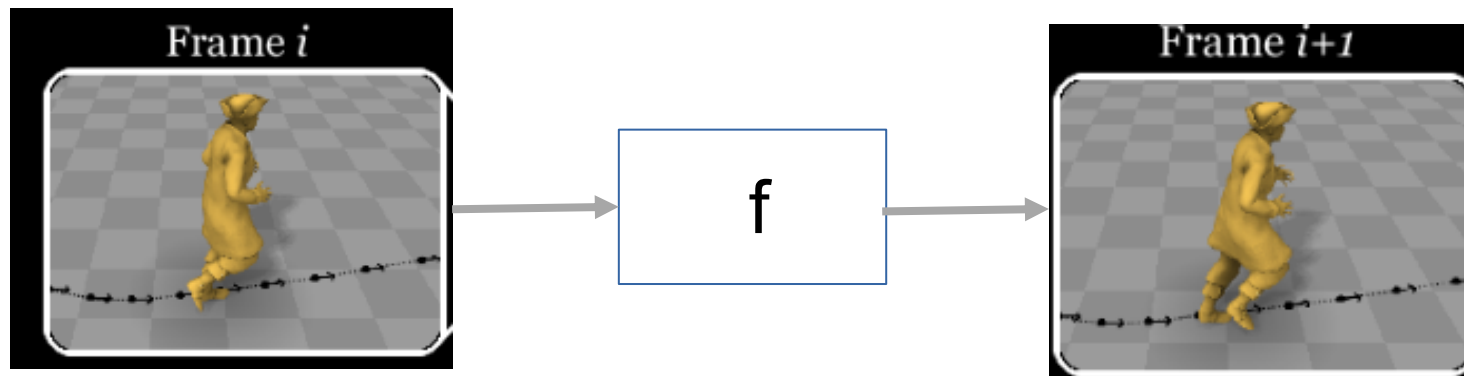


Figure 6: A boxing graph.

Deep Time Series Models

- Predicting the next state from the current state and control signals
 - $S_{t+1} = f(S_t, C_t)$
- cRBM, LSTM, etc.

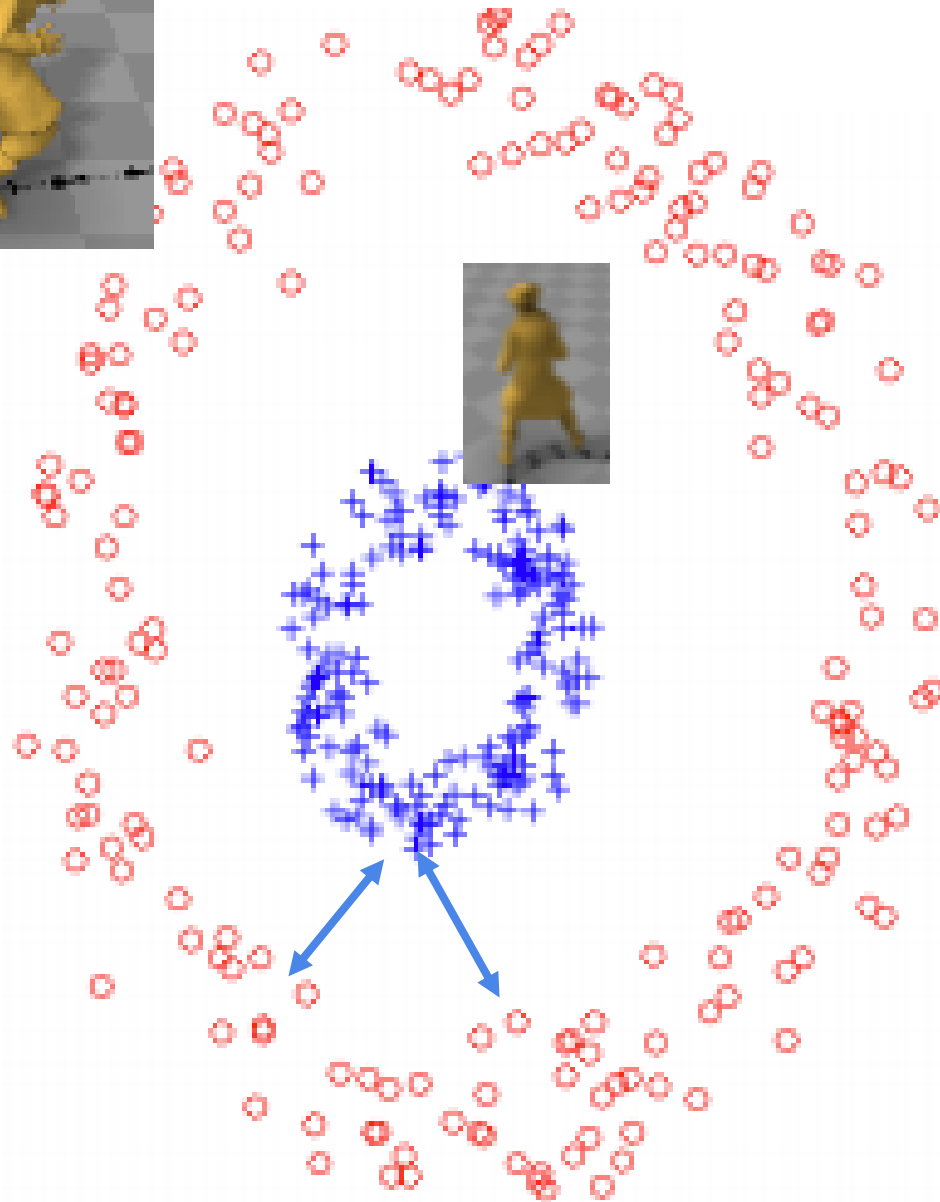


Deep Time Series Models: Issues

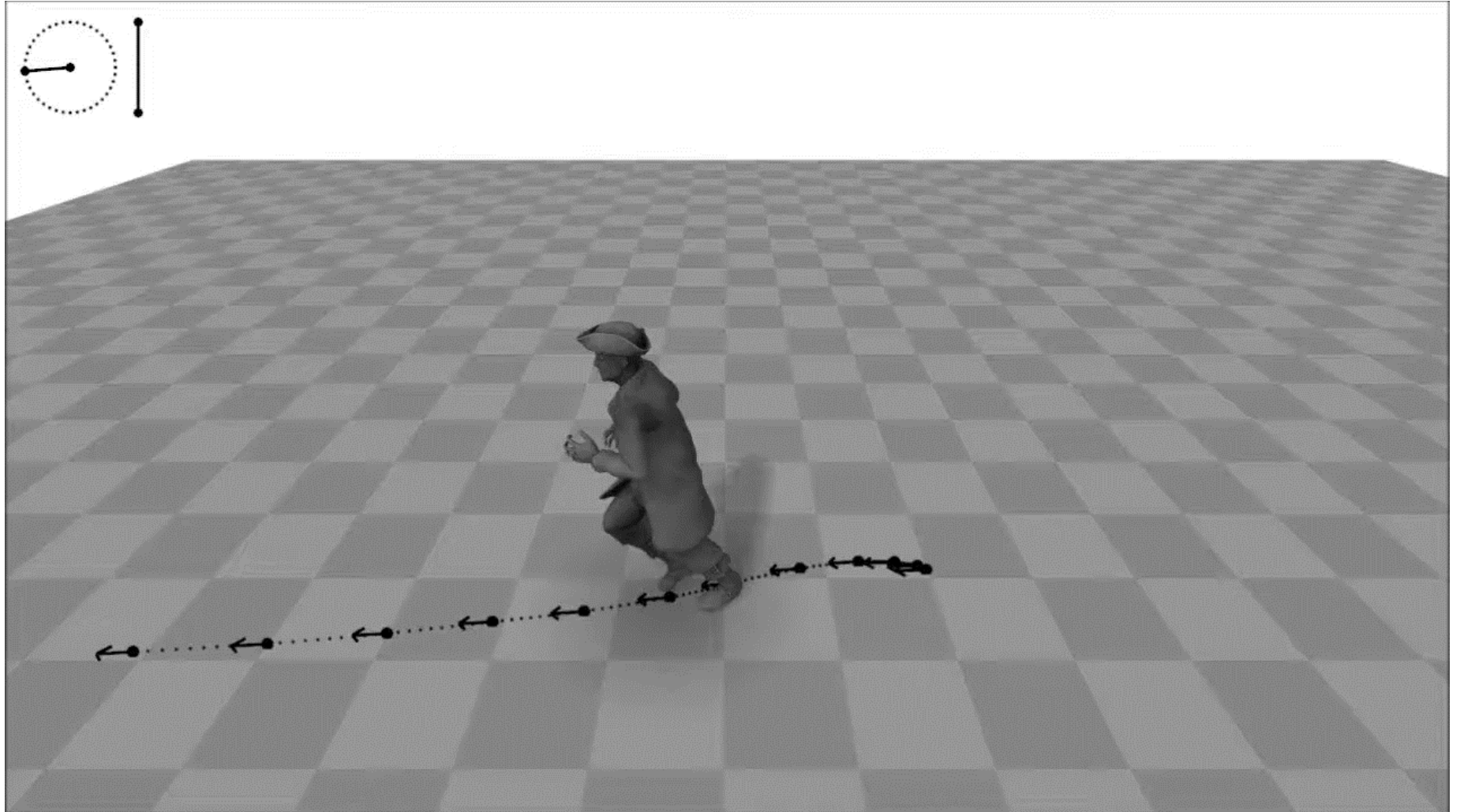
- Ambiguity in the future
 - Wide variation of motions can happen in the future!
 - Converging to average poses
 - Motion smoothed out

Deep Time Series Models: Issues with Human Locomotion

- The transition between motion is very sparse in human motion
- Think of switching from side stepping to running
 - This rarely happens and people usually do it in only one way

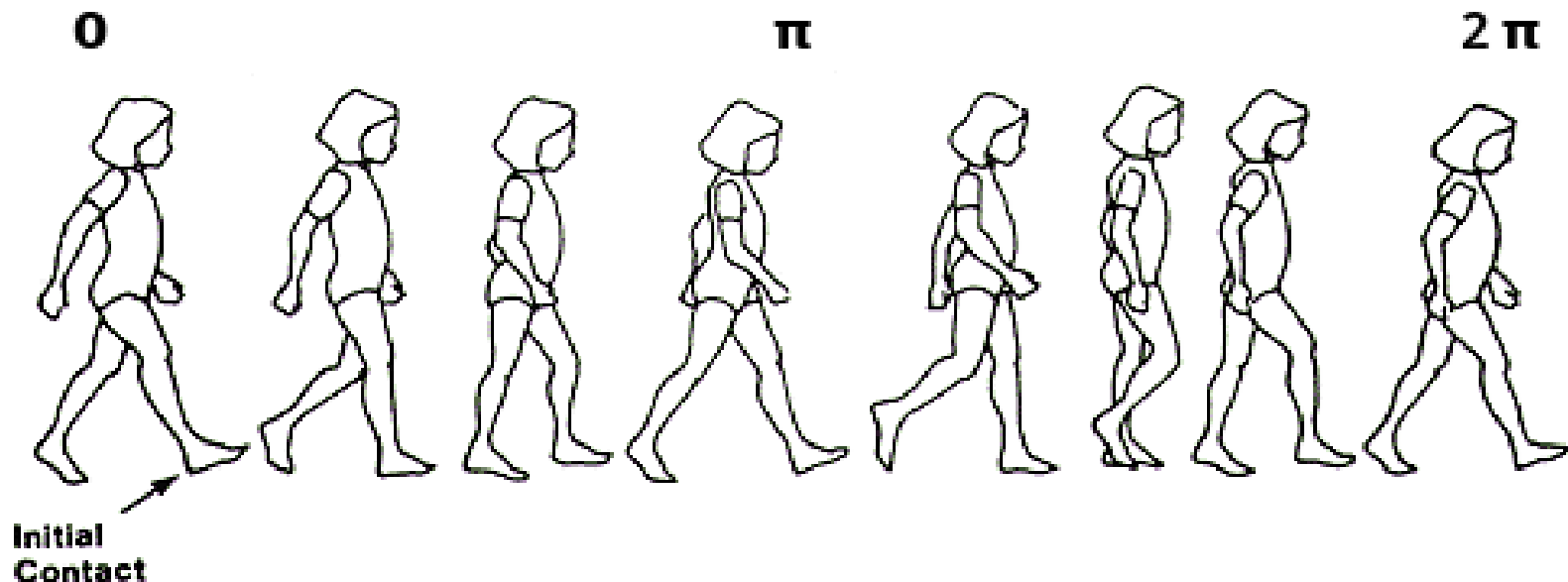


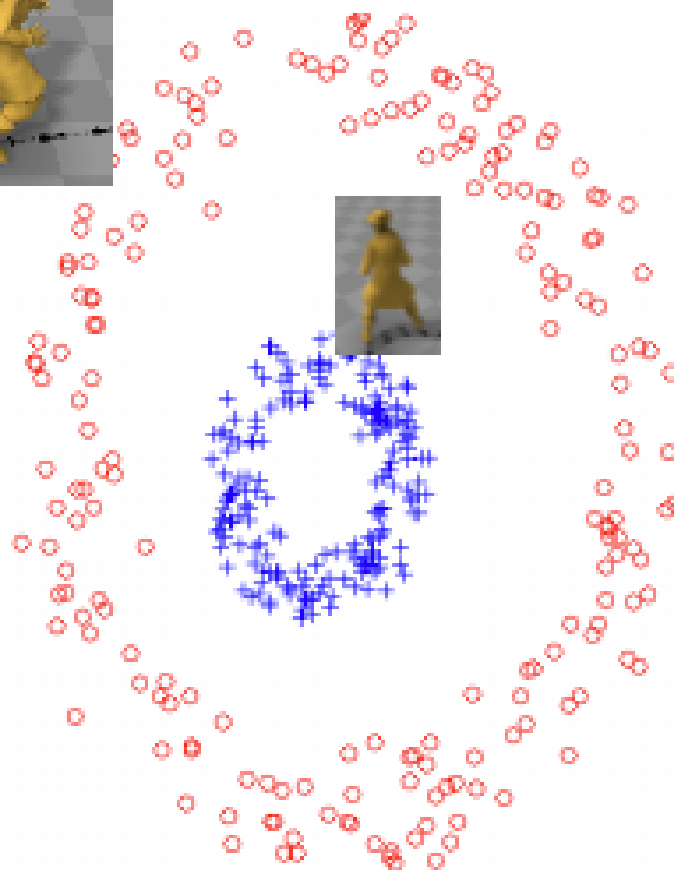
Example of a smoothed out animation

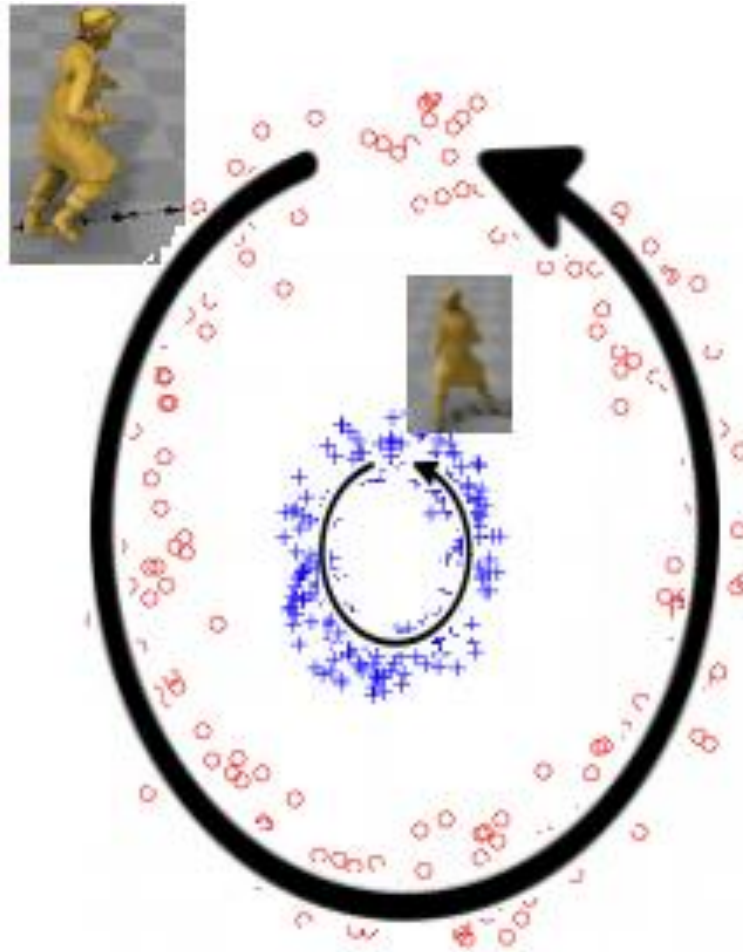


Phase Label

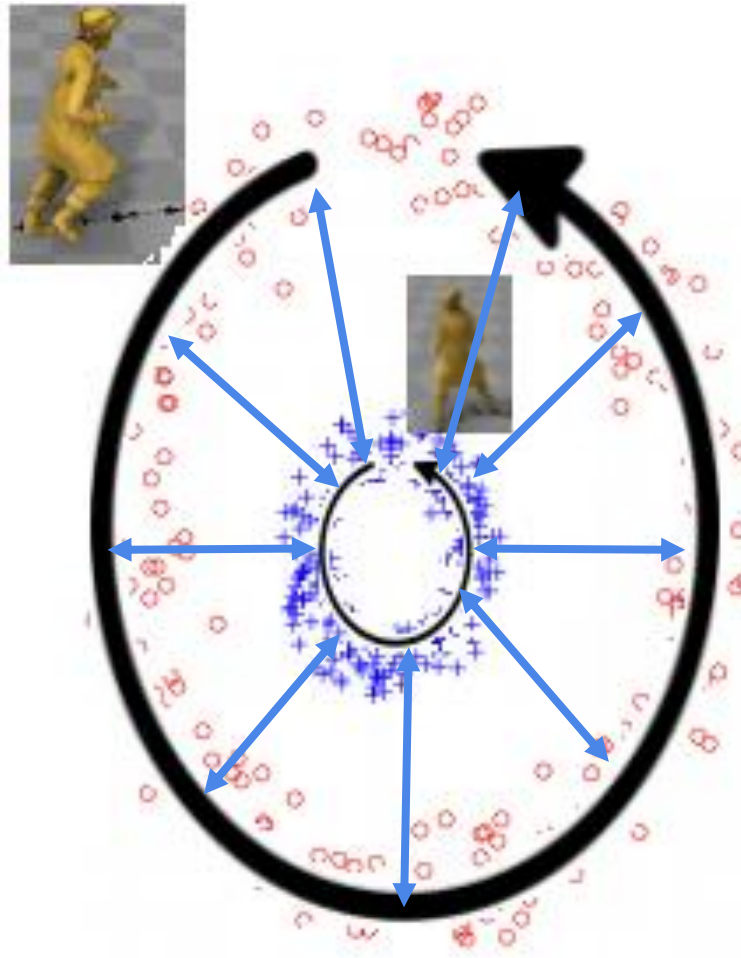
- Changes from 0 to 2π
 - 0 when right foot contacts the ground
 - π when the left foot contacts the ground
 - 2π when the right foot contacts the ground again



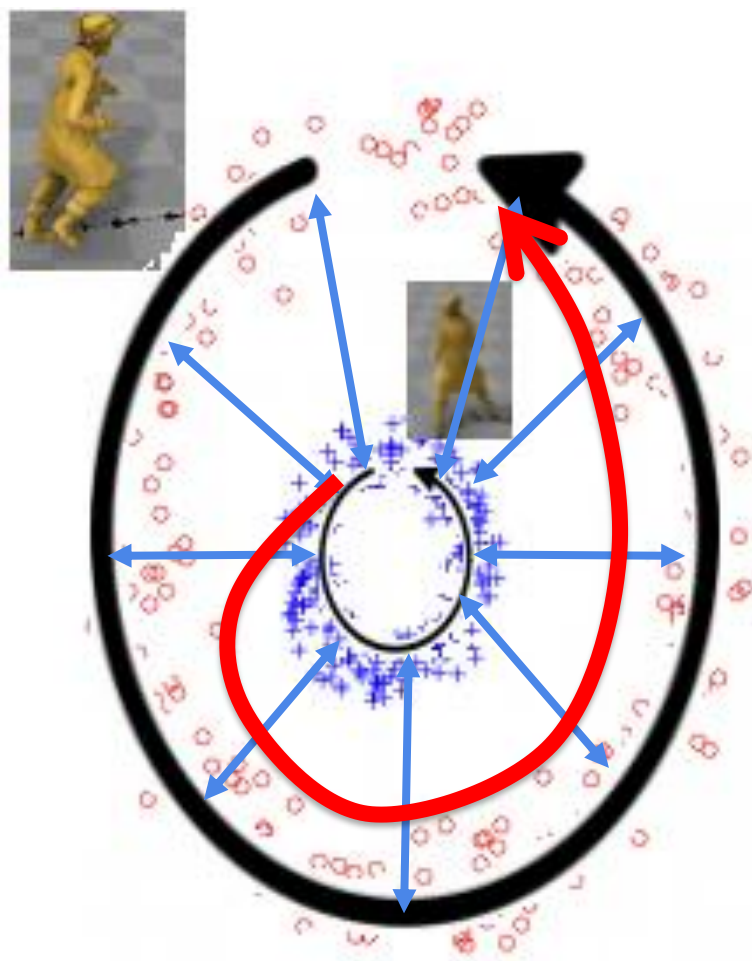




Phase
label



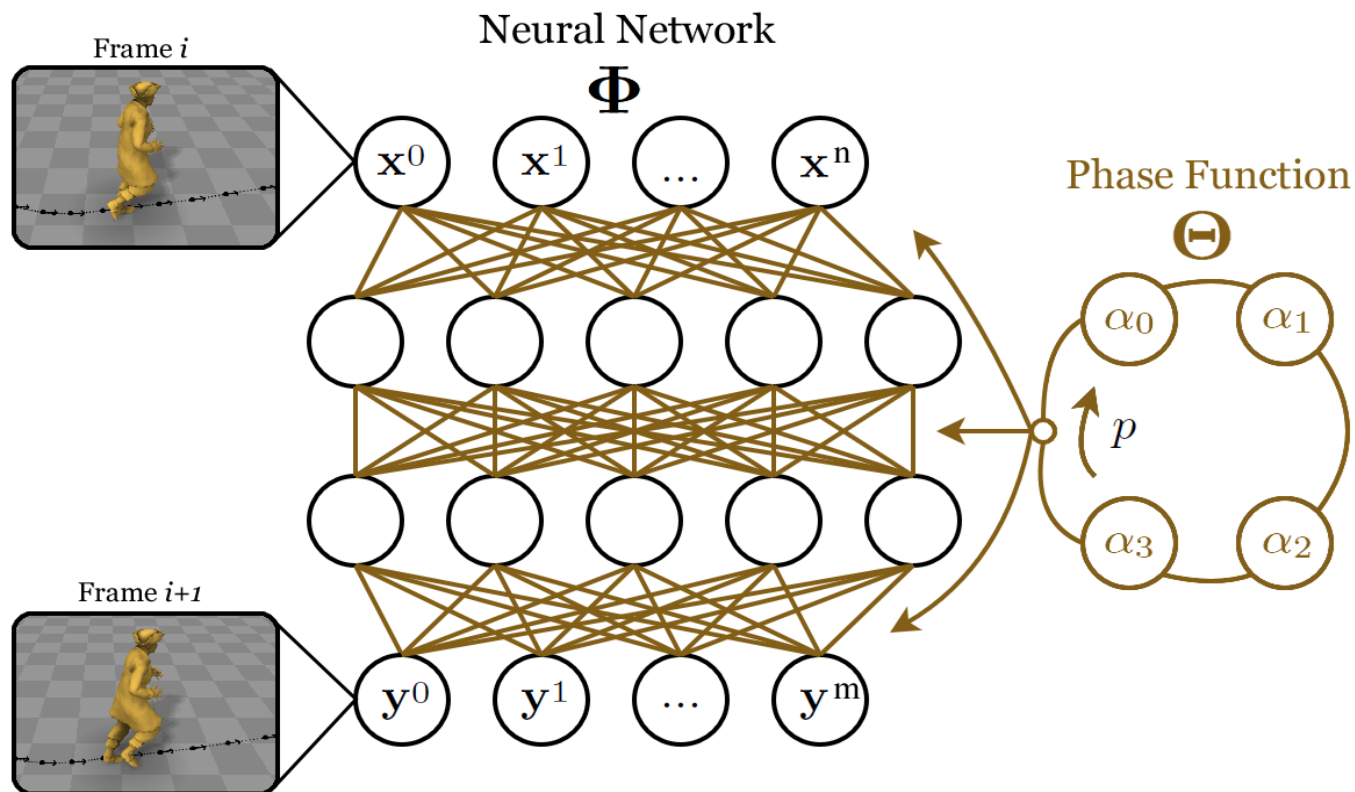
Phase
label



Phase-Functioned Neural Networks (PFNN)

Holden et al. 2017

- The network weights are functioned by the “phase” of the gait



Inputs/Outputs

Input:

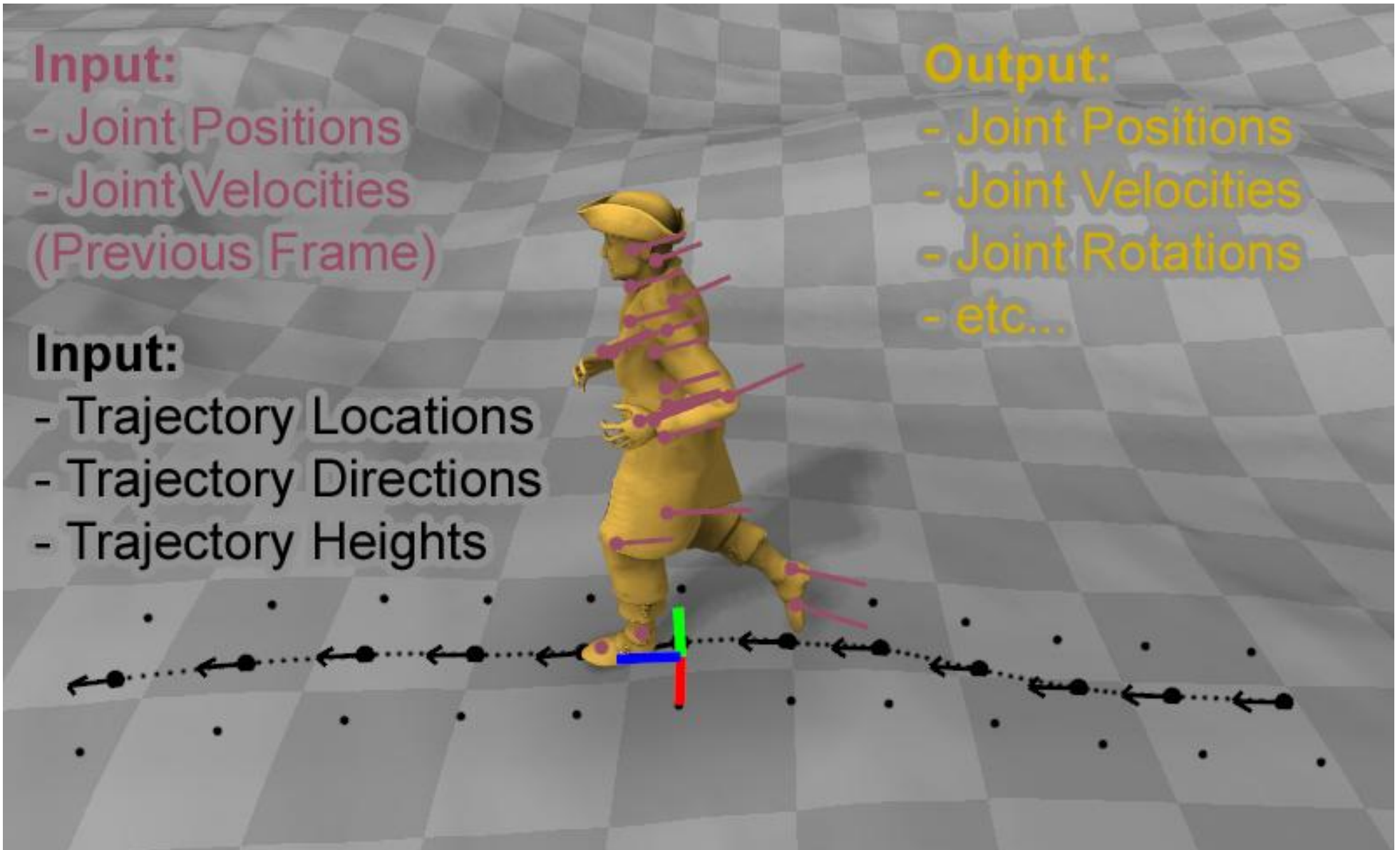
- Joint Positions
- Joint Velocities
- (Previous Frame)

Input:

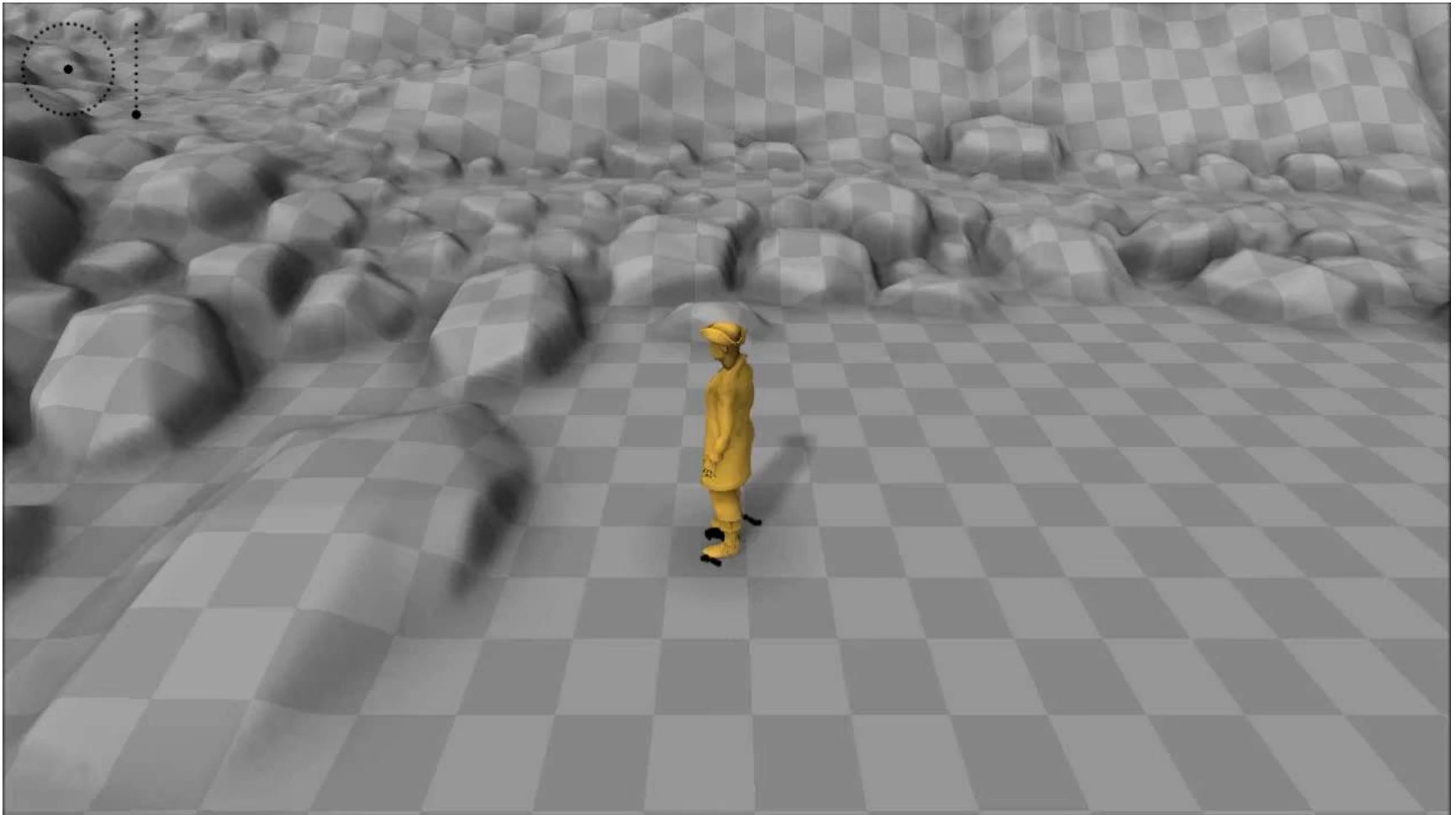
- Trajectory Locations
- Trajectory Directions
- Trajectory Heights

Output:

- Joint Positions
- Joint Velocities
- Joint Rotations
- etc...



PFNN Demo



Urban Environment



[https://github.com/sebastianstarke/Al4Animation
?tab=readme-ov-file](https://github.com/sebastianstarke/Al4Animation?tab=readme-ov-file)

015
000
250000

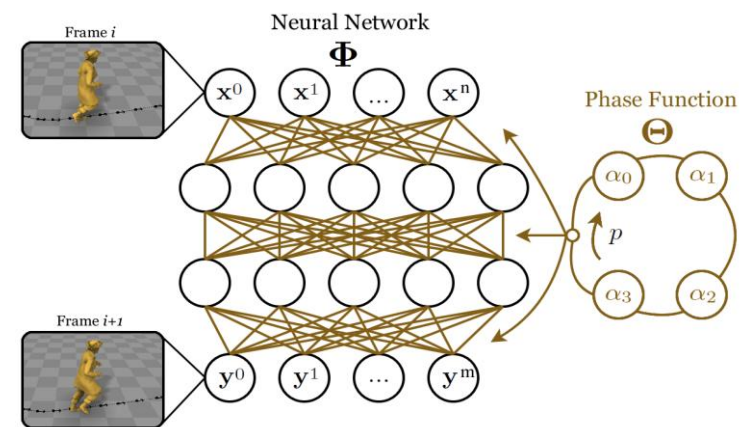
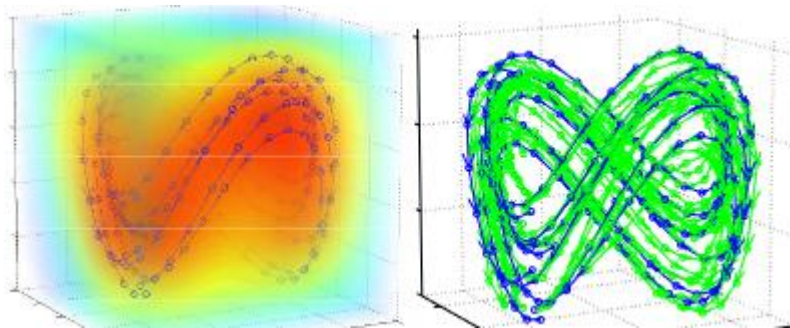


Discussions

- Giving the phase
 - helps to remove the ambiguity
 - helps to produce transitions between different motion styles

Discussions (2)

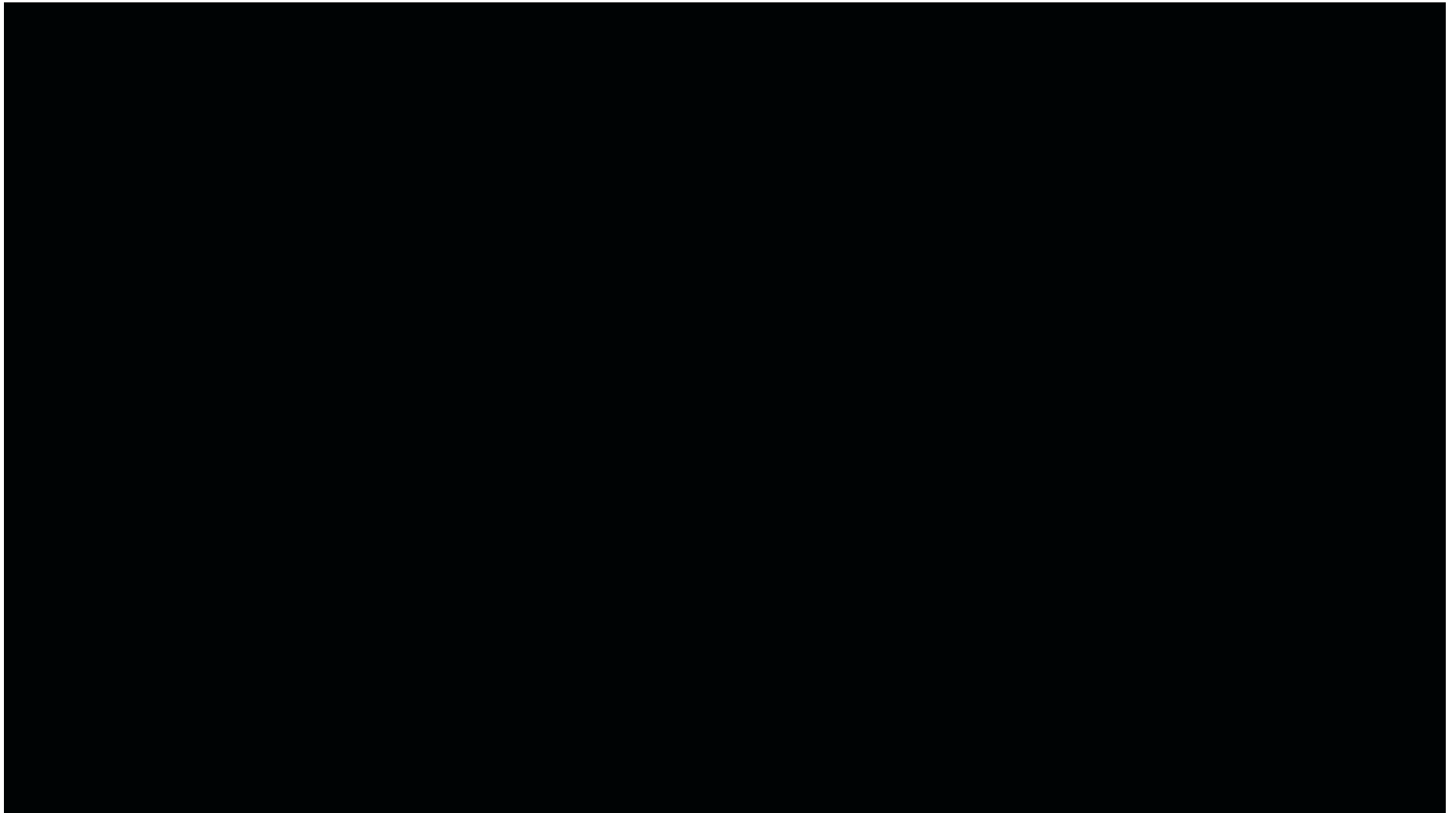
- An architecture that considers the structure of the data is helpful for learning
- Especially for producing high quality motion with smooth transitions



Discussions (3)

- But the phase labels must be added to the data
- How about other type of movements?
 - Like sitting, grabbing objects etc.

Some more extensions



Some more extensions



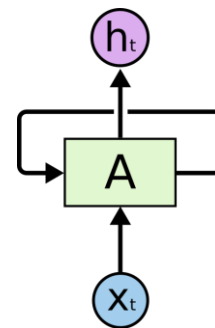
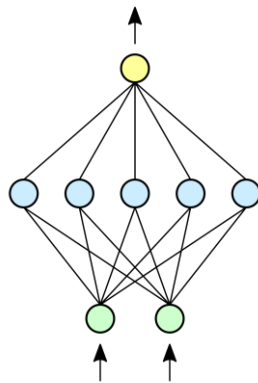
Discussions

- Phase Functioned Neural Network can produce natural transitions between motions but requires adding the phase labels

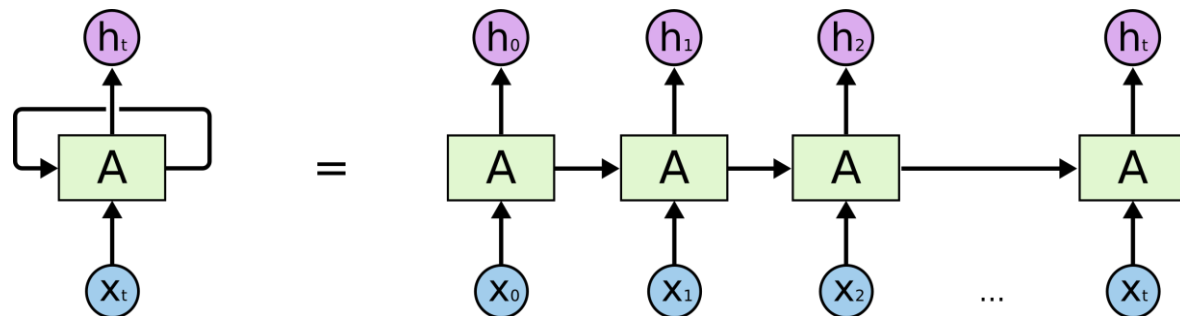
Recurrent Neural Networks (RNN) Long Short Term Memory (LSTM)

Neural networks used in Computational Linguistics for language understanding

They are networks with loops in them, allowing information to persist.



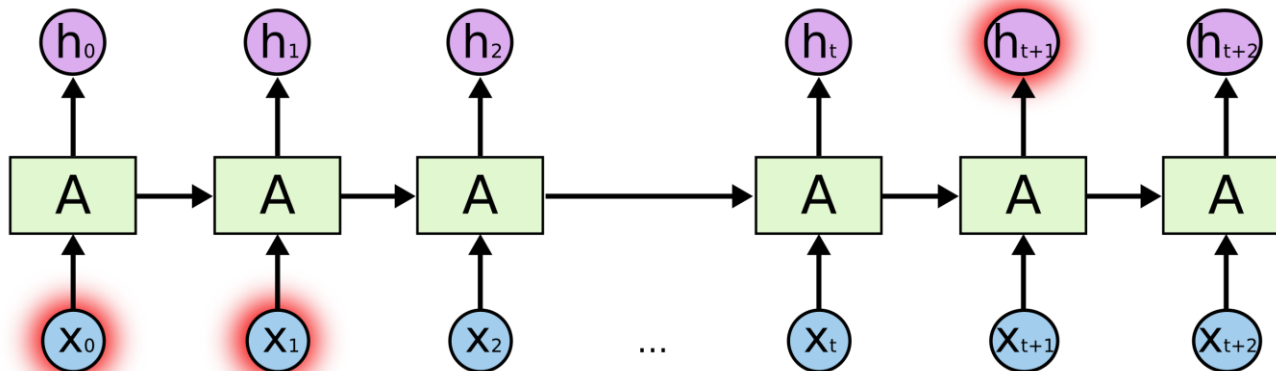
- The network looks at some input x_t and outputs a value h_t .
- A loop allows information to be passed from one step of the network to the next.
- A recurrent neural network can be thought of as multiple copies of the same network, each passing a message to a successor.



Problem of Long-Term Dependencies

- RNNs have issues remembering information long from the past
- For example, when predicting the final word in the sentence

“I grew up in France. I can speak *French*.” - this is difficult with RNNs.



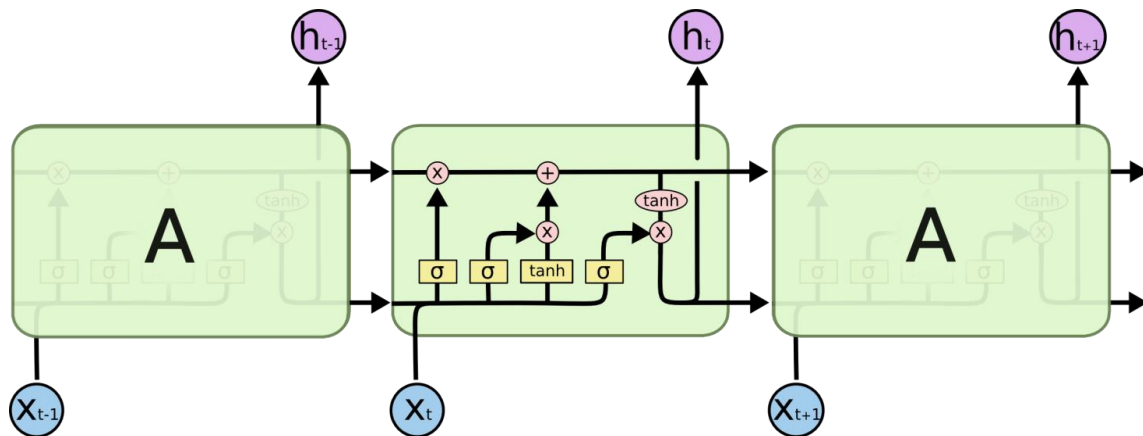
LSTM

LSTM is an architecture developed to improve this – widely used in language models and time-series modelling.

LSTMs are designed to avoid the long-term dependency problem.

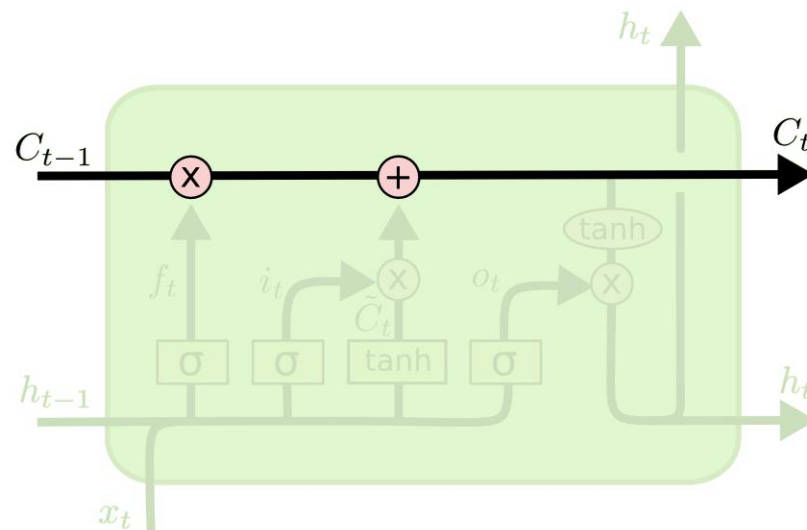
Remembering information for long periods of time is their default behavior

LSTM has gates that decides which information should be kept and which should be forgotten.



Internal State

- The key to LSTMs is the cell (internal) state, the horizontal line running through the top of the diagram.
- The cell state is kind of like a conveyor belt.
- It runs straight down the entire chain, with only some minor linear interactions.
- It's very easy for information to just flow along it unchanged.

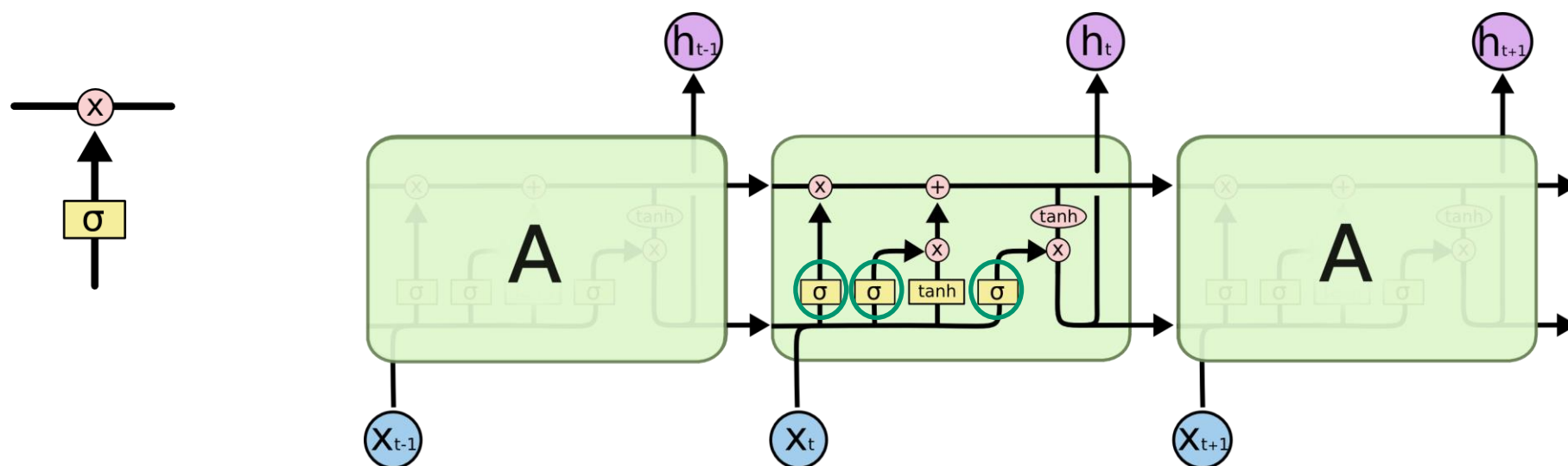


Gating

The sigmoid layer outputs numbers between zero and one, describing how much of each component should be let through.

A value of zero means “let nothing through,” while a value of one means “let everything through”

An LSTM has three of these gates, to protect and control the cell state.



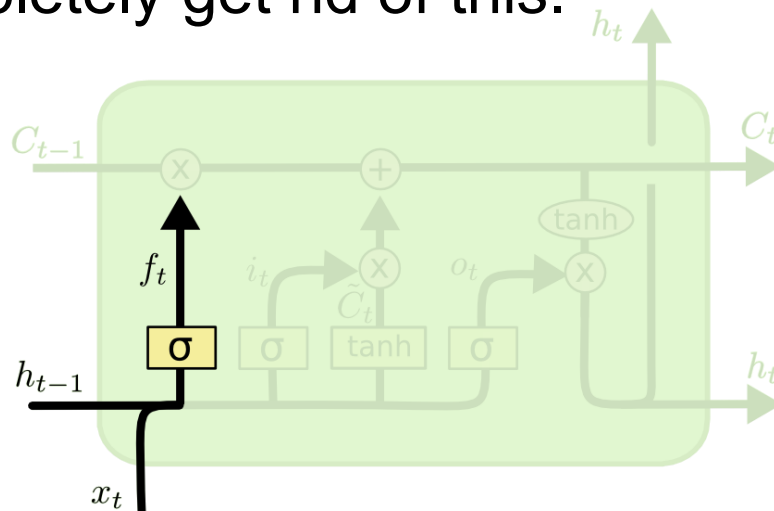
Forget Layer

The first step is to decide what information we're going to throw away from the cell state.

This decision is made by a sigmoid layer called the “forget gate layer.”

It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} .

A 1 represents “completely keep this” while a 0 represents “completely get rid of this.”



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

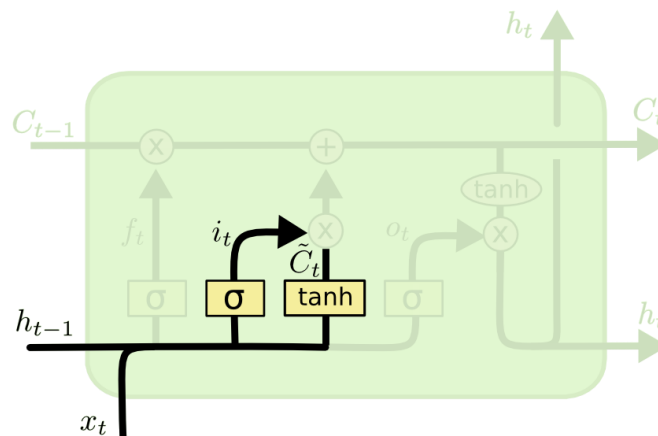
Input gate

The next step is to decide what new information we're going to store in the cell state.

This has two parts.

First, a sigmoid layer called the “input gate layer” decides which values we'll update.

Next, a tanh layer creates a vector of new candidate values, $\tilde{C}$ that could be added to the state.



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

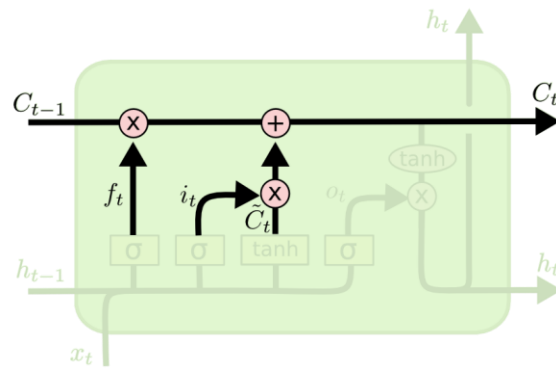
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Update gate

It's now time to update the old cell state, C_{t-1} , into the new cell state C_t .

We multiply the old state by f_t , forgetting the things we decided to forget earlier.

Then we add $i_t * \tilde{C}_t$



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

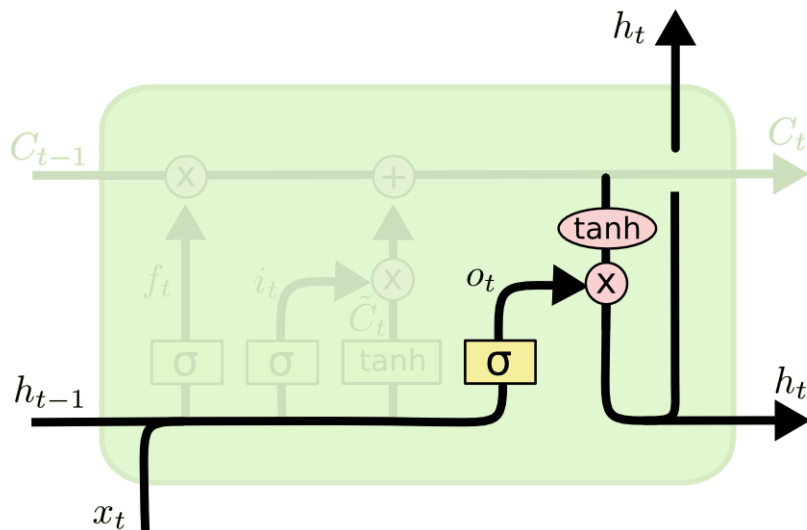
Output

Finally, we need to decide what we're going to output.

This output will be based on our cell state, but will be a filtered version.

First, we run a sigmoid layer which decides what parts of the cell state we're going to output.

Then, we put the cell state through tanh (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.



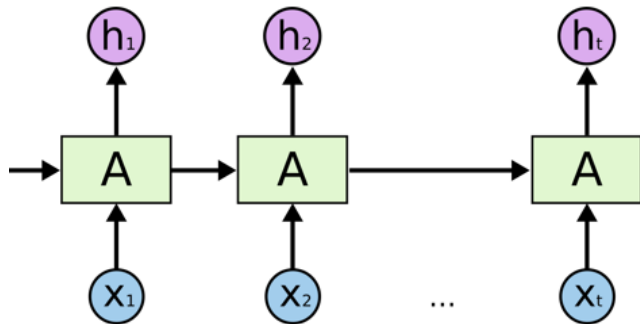
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Application of LSTM to human motion

We can use LSTM as a time series model

People have been applying it for locomotion synthesis, basketball animation



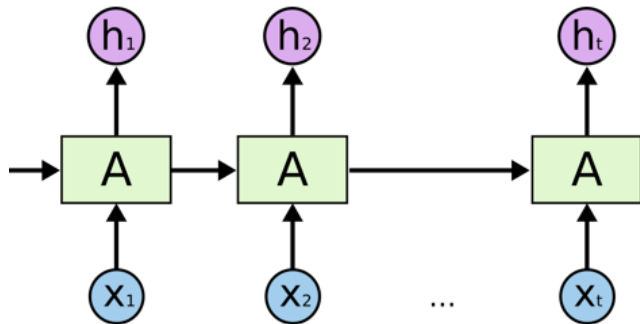
Lee et al. 2018, "Interactive Character Animation by Learning Multi-Objective Control"

Application of LSTM to human motion: Issues

How to initialize the hidden units?

The response to inputs can be very slow.

Why?



Lee et al. 2018, "Interactive Character Animation by Learning Multi-Objective Control"

LSTM



NSM (ours)



Summary

- Time-series models are useful for real-time motion synthesis.
 - Phase-Functioned Neural Network
 - Periodic Autoencoder
 - LSTM
- We need to provide a good control signal and design an architecture to avoid the ambiguity problem
- When designing the architecture, it is helpful to consider the structure of the data
- LSTMs are good, but can have issues when designing real-time character controllers.

Readings

Understanding LSTM Networks

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Holden et al. “Phase-Functioned Neural Networks for Character Control”, SIGGRAPH2017

Starke et al. 2022, “DeepPhase: Periodic Autoencoder for Learning Motion Phase Manifolds”

Lee et al. 2018, “Interactive Character Animation by Learning Multi-Objective Control”